

Projet Tower Defense

Objectif : Réaliser intégralement un jeu de type Tower-Defense avec Unity.



WANG Julien

GONZALEZ Alexandre

FRUCHET Antonin

Introduction

TowerDefense est un jeu de type Tower Defense (comme son nom l'indique) dans un style cartoonesque. Des vagues de monstres apparaissent et tentent de traverser la carte pour endommager vos cristaux. C'est pourquoi vous vous défendez à l'aide de diverses tours. Pour s'en sortir, la stratégie doit être le maître-mot : à court d'énergie ou à court d'argent, c'est la fin !

Ce projet est pour nous trois le plus gros projet de développement que nous ayons réalisé à cette date.

Présentation de l'application / description détaillée des fonctionnalités de l'application.

Reprenons le cahier des charges point par point, et détaillons comment les fonctionnalités ont été implémentées.

Chargement et Rendu d'une Carte

Le dossier Maps se trouve dans le *persistantDataPath*

- Pour Windows : *AppData/LocalLow/{Company}/{ProjectName}/Maps*
- Pour MacOS : *Library/Application Support/{Company}/{ProjectName}/Maps*

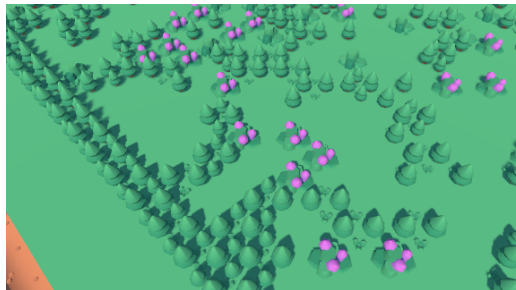
Ce dossier se crée au lancement du jeu s'il n'existe pas, mais aucune carte ne s'ajoute automatiquement, il faut donc les mettre à la main.

Chargement et visualisation d'une carte à partir du dossier Maps

Lors de la sélection d'une carte dans le menu, celle ci est lue avec System.IO, pixel par pixel, transformée en Texture2D, puis transformée en tableau de type *TileType[][]*, *TileType* étant une énumération qui représente le type de case (chemin, intersection, debut, fin, constructible et bord) avec une correspondance de couleur. Ce tableau est chargé dans la classe statique *GameConfig* prêt à être récupéré par le moteur de jeu pour générer la carte.

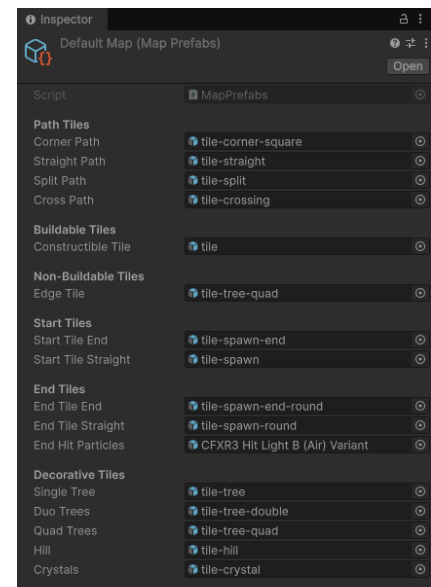
Une fois la scène de jeu lancée, *Game* récupère son composant *MapGenerator* et appelle *void GenerateMap()* qui instancie les bonnes tuiles au bons endroit avec les bonnes rotations en fonctions des chemins voisins notamment (cf le script MapGenerator.cs sous Assets/Scripts qui est bien commenté).

Bonus : *void GenerateMap()* appelle *void GenerateBorders()* à la fin de son exécution ce qui génère un bout de monde hors des limites du jeu ce qui cache le fond gris de Unity.



Bordures

Bonus 2: Les prefabs composant la carte sont changeables dans des assets (avec un ScriptableObject). En créant plusieurs assets de prefabs différents on peut faire des styles de cartes différents. C'est pourquoi le menu permet, à l'aide d'une case à cocher, de changer le style de la carte par défaut dans un style **hivernal**.



Il est possible de rajouter une carte dans le dossier Maps pendant que le jeu est lancé, un bouton "*refresh*" doit dans ce cas être cliqué pour actualiser les changement dans le dossier.

Création du graphe des chemins

Pour créer un graphe, il faut d'abord construire une représentation de graphe. Ainsi, un graphe dans cette application sera modélisé par la classe générique *Graph<T>* et ses arêtes par la classe génériques *Vertex<T>* (*Graph.cs*). T sera ici le type *VertexLabel* qui est une enum représentant les types de nœuds du graphe (START, END, INTERSECTION). Un graphe est donc un ensemble de sommets liés à leurs voisins par une distance.

Pour associer les sommets à leur voisins, on part d'un sommet du graphe, on regarde sur la carte en *TileType[][]* de quel côté de ce sommet il y a un chemin (ou pas), s'il y en a un, on avance dans cette direction (rectilignement car les chemins doivent être rectilignes) jusqu'à trouver un sommet en comptant la distance et ce pour toutes les directions où il y a un chemin. Le sommet trouvé devient le nouveau sommet de départ et on applique récursivement ce processus jusqu'à ce que tous les sommets aient été visités. Pour tenir compte de graphes potentiellement non connexes, on fait partir l'algorithme de chaque sommet du graphe en gardant la trace des sommets visités d'une exécution à l'autre. Il est possible que lors de l'exécution de cette procédure un chemin partant d'un sommet soit trouvé mais que celui ne termine pas pas rectilignement sur un autre sommet. À ce moment, une exception expliquant le problème est levée.

La classe utilitaire *PathVerifier* implémente cela avec la méthode *void CreatePathGraph()*.

Vérification de la jouabilité de la carte

Deux conditions sont nécessaires à la jouabilité d'une carte :

- La carte doit présenter au moins un début et une fin
- Chaque début doit être relié à une fin.

Pour se faire la classe ***PathVerifier*** implémente une méthode *void IsValidGraph()* qui vérifie ces deux propriétés. Dans le premier cas la vérification est triviale, dans le deuxième il suffit de regarder tous les débuts et de faire un parcours de graphe en profondeur à partir de chacun d'eux pour trouver un sommet de fin.

Si l'une de ces conditions est violée *void IsValidGraph()* lève une exception qui explique la violation.

À cela s'ajoute l'exception du point précédent qui constitue aussi une carte injouable.

Contrôleur de caméra permettant de se déplacer sur la carte

Le contrôle de la caméra est fait avec les *InputAction* comme dans le TP2 avec les vases, *PlayerController* fournit les input que le script *CameraControle* utilise pour gérer les fonctionnalités. Nous avons la possibilité de bouger notre *POV (Point of view)* avec les touches ZQSD et également avec la fonction drag du clic droit. Le zoom peut être effectué de deux manières différentes; soit sur clavier avec "O" pour zoom out et "P" pour zoom in, soit tout simplement avec la molette.

Développement du jeu

Gestion du déroulement du jeu

Un cycle de jeu se déroule en deux phases :

- Phase de Préparation : Il n'y a aucun spawns d'ennemis, le joueur peut acheter, placer, déplacer, vendre des bâtiments autant que son porte-monnaie le lui permet. Il dispose pour cela du temps qu'il veut et c'est lui qui lance la prochaine phase quand il est prêt.
- Phase de Défense : Une vague d'ennemis apparaît à chaque point de spawn. Le joueur ne peut plus toucher aux bâtiments qu'il a placés sur la carte à la phase précédente. Il faut que le joueur tue des ennemis pour gagner de l'argent. Ainsi il pourra ajouter plus de bâtiments au prochain tour. La phase se termine quand tous les ennemis sont morts (ils meurent forcément car quand ils arrivent à la fin de leur parcours ils tapent le cristal et meurent sur le coup).

Le cycle préparation / défense est géré par *Game* qui possède un champ *public state* en lecture seule qui prend les valeurs *GameState.Preparation* et *GameState.Defense*. Les Actions possibles du joueur sont donc restreintes ou pas en fonction de ce paramètre directement dans *PlayerController*.

Une partie se termine quand les points de vie du joueur tombent à zéro.

Ajout, suppression de bâtiments

L'ajout (ou plutôt achat) de bâtiments se fait par l'intermédiaire d'un petit menu roue apparaissant au clic sur un emplacement libre. Le bâtiment de son choix peut être placé uniquement sur une tuile constructible qui n'est pas déjà occupée.

Une fois des bâtiments placés, il est possible de les déplacer ou de les supprimer. Pour ce faire, on peut cliquer sur le bâtiment d'intérêt et un petit menu avec deux boutons apparaît : déplacer ou recycler. Si on décide de recycler le bâtiment, l'objet est détruit et son prix moitié remboursé par l'intermédiaire de la méthode *SellBuilding()* de *Building*.

Quand on déplace le bâtiment, celui-ci est d'abord désactivé, surélevé, attaché à la position de la souris, puis un *material* transparent vert ou rouge est appliqué en fonction de la plaçabilité du bâtiment à cet endroit. On peut ensuite cliquer pour replacer le bâtiment à un nouvel endroit. C'est le script *BuildingPlacementManager* qui gère cela.



à gauche. position possible. à droite. position impossible

Spawn et déplacement des monstres

Les monstres se déplacent avec un composant *CharacterController*. Ils ont deux stratégies pour rejoindre un point d'arrivée :

- Plus court chemin : Algorithme de Dijkstra
- Plus court chemin + recouvrement minimal : Algorithme de Dijkstra avec des distances modifiées pour prendre en compte le risque de passer par un chemin ou plein de tours peuvent tirer.

Nous nous sommes permis de ne pas implémenter le recouvrement proposé dans le cahier des charges car si un chemin n'est pas recouvert, son poids sera automatiquement 0, n'importe quel chemin sera emprunté.

Dans les deux cas, il est possible que plusieurs chemins de poids équivalents soient trouvés et que le premier soit toujours choisi. On a donc récupéré tous ces chemins au lieu d'un seul et le monstre en choisit un aléatoirement.

Ces stratégies sont implémentées dans les méthodes de la classe utilitaire *Strategies* et un champ sérialisé sur les monstres permet de leur attribuer rapidement une stratégie ou une autre.

Détails des types de monstres :



De gauche à droite : GroBleu, GroJaune, Blob, Shell

Ces monstres ont des caractéristiques différentes :

Monstre / Statistique	Vie	Dégâts	Vitesse	Stratégie	Argent rapporté
GroBleu	10	3	moyenne	PCCR	25
GroJaune	10	3	moyenne	PCC	25
Blob	80	20	lente	PCC	100
Shell	5	1	moyenne	PCCR	20

PCC : Plus Court Chemin, PCCR : Plus Court Chemin Recouvrement

Lors de la phase de Défense, une vague de monstres apparaît. Cette vague est générée presque par le script *WaveManager*. Ce script génère des vagues comme des listes de *MonsterType* et une coroutine d'instanciation des monstres est appelée quand l'état du jeu passe à *GameState.Defense*. La vague apparaît à un point de spawn aléatoire parmi ceux présent sur la map.

Voici un tableau qui détaille la forme des vagues au fil de l'avancée de la partie (🐚 = Shell, 🦾 = Grobleu ou GroJaune aléatoirement, ● = Blob) :

Vague	Tête de file	Milieu	Fin de file
1	🐚 ×1	🦾 ×1	🐚 ×1
2	🐚 ×2	🦾 ×1	🐚 ×2
3	🐚 ×4	🦾 ×1	🐚 ×4
4-5	🐚 ×5	🦾 ×2	🐚 ×5
6-8	🐚 ×5	🦾 ×2 ● ×1 🦾 ×2	🐚 ×5
9-11	🐚 ×5	🦾 ×2 ● ×2 🦾 ×2	🐚 ×5

12-14	👤 ×5	💪 ×2 ● ×3 💪 ×2	👤 ×5
15-17	👤 ×5	💪 ×2 ● ×4 💪 ×2	👤 ×5
18+	👤 ×5	💪 ×2 ● ×5+ (*) 💪 ×2	👤 ×5

(*) Un Blob de plus apparaît tous les 3 tours.

La vie des monstres est visible grâce à une petite barre de vie flottante au-dessus de leur tête.



Barre de vie des monstres

Cette barre est un simple UIDocument que le monstre porte sur lui. La position de la barre suit le monstre en temps réel grâce à la méthode *TrackPosition()* du composant *MonsterHealthbar*. On y utilise notamment la fonction *WorldToScreenPoint()* pour transformer la position du monstre en position 2D à l'écran.

Gestion des actions des bâtiments

Les bâtiments ont chacun leur(s) action(s) propre, et les gèrent eux même dans leur scripts.

- Les tours (*Tower*), tournent sur elles-même de manière aléatoire quand elles n'ont pas de cible. La classe *Tower* est abstraite et force les méthode *Shoot()* et *UpdateTarget()* à être implémentées chez les classe filles. Ainsi, chaque tour actualise sa cible d'une manière personnalisée (les comportements sont implémentés dans *TargetStrategies.cs*), et tire de manière personnalisée.
- Les installations elles sont détectées par les tours qui actualisent leur statistiques

- La centrale d'énergie n'effectue aucune action si ce n'est fournir un champ public de la valeur de la puissance générée.



De gauche à droite : InfernoTower, Gun, X-Bow, Canon

Tours / Stats	InfernoTower	Gun	Xbow	Canon
Dégats	1	4	2	5
Portée	1	2	4	3
Cadence	3	4	3	1
Ciblage	Multi	Mono	Mono	Mono
Consommation énergétique	3	3	3	3



De gauche à droite : Factory, Radar, Storage, PowerPlant

Visualisation des tirs des tours

Pour trois des quatre tours, Canon, Gun et X-Bow les projectiles sont des Rigidbody (solution la plus simple à implémenter bien que très peu efficace en réalité, car les projectiles ratent souvent leur cible)

La dernière tour, Inferno Tower, ne tire pas de projectile mais plutôt émet un laser (représentés avec un *LineRenderer*) qui infligent des dégâts réguliers à toutes les cibles dans sa portée.



InfernoTower attaquant des monstres

Gestion de l'énergie

Chaque centrale d'énergie délivre une puissance de 7 unités, et chaque tour en consomme 3 unités (les installations ne requièrent pas d'alimentation).

L'attribution de l'énergie aux tours se fait en deux étapes avec le script (*EnergyManager*) :

1. Création du graphe de l'énergie : Le graphe sera modélisé par la classe *Graph<Building>*. D'abord on récupère tous les bâtiments de type *Tower* et *PowerPlant* qu'on ajoute comme sommets du graphe. Pour les relier, on applique un algorithme de parcours en largeur multisource à partir de chaque centrale d'énergie. Les sommets sont donc explorés couche par couche et reliés à des voisins de manière à ce que chaque tour soit le plus proche d'une centrale possible. (deux sommets voisins d'une centrale ne sont pas voisins entre eux car déjà reliés à la centrale, et une tour à distance d'une tour alimentée et d'une centrale sera liée à la centrale). Ainsi construit, le graphe d'énergie n'est qu'un ensemble d'arbres disjoints dont les racines sont les centrales. Le code est plutôt bien commenté dans la méthode *ConnectBuildings()* de *EnergyManager.cs*.
2. Répartition de l'énergie : Pour répartir l'énergie au sein d'un arbre (une centrale et ses tours connectées), nous avons choisi une répartition équitable entre toutes les tours de l'arbre. Comme nous travaillons avec des unités, (des *int*) un petit travail de division euclidienne et de répartition du reste est nécessaire. (cf méthode *SetEnergyToBuildings()* de *EnergyManager.cs*). Ceci dit, il est facile de créer une autre méthode *SetEnergyToBuildings()* qui attribuait l'énergie plus finement.

Une tour ne peut donc pas tirer si elle n'est pas assez alimentée. Pour visualiser l'alimentation de chaque tour et la puissance fournie par les centrales, un petit élément d'UI indique l'alimentation actuelle des tours et la puissance fournie par la centrale. Des lignes bleues sont aussi rendues pour montrer les connexions d'énergie entre les bâtiments concernés.



Démonstration des indicateurs d'énergie.

De la même manière que pour les barres de vies des monstres, les indications d'alimentation sont des petits UIDocument que tours et centrales portent sur eux. La structure du script est sensiblement la même que celle de l'affichage des barres de vies de monstres.

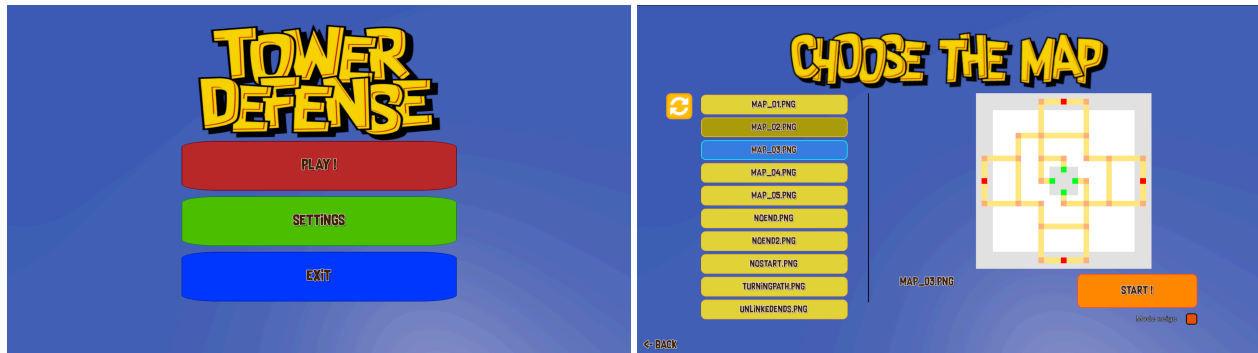
Menus pour choisir une carte et lancer une partie

Le menu principal possède une page de sélection de la carte et de lancement d'une partie. Celle-ci affiche toutes les cartes du dossier Maps sous forme de boutons dans un *ScrollPanel*.

Un bouton pour actualiser est aussi présent dans le cas où le dossier *Maps* est modifié.

Quand on clique sur un bouton de carte, est affiché une prévisualisation de la carte (pas en 3D mais on affiche juste l'image de la carte), le nom de la carte, une erreur adaptée si la carte n'est pas conforme ainsi qu'un bouton de lancement de la partie qui n'est actif que si la carte sélectionnée est conforme.

Une petite case à cocher permet aussi de changer le style de la carte en mode hivernal comme évoqué plus tôt.

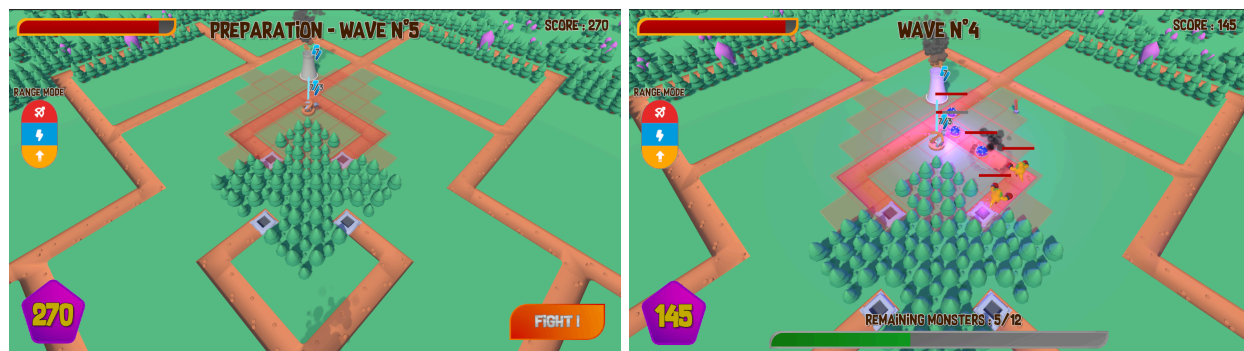


Menu principal et sélection des cartes

Système de paramètres sauvegardés dans les PlayerPrefs Unity

PlayerPrefs fonctionne comme un dictionnaire avec un système de (clé, valeur). Les clés sont des *string* et les valeurs peuvent être de trois types : *int*, *float* ou *string*. Dans notre cas c'est amplement suffisant car nous l'utilisons pour sauvegarder : *Brightness(float)*, *Sound(float)*, *Music(float)*, *Master(float)*. (pour respectivement la luminosité, le volume des effets sonores, le volume de la musique, et le volume général)

Interface Utilisateur en jeu (HUD)



HUD Global, Préparation à gauche et Défense à droite

Un HUD personnalisé est présent dans le jeu et permet une meilleure interaction joueur-jeu.

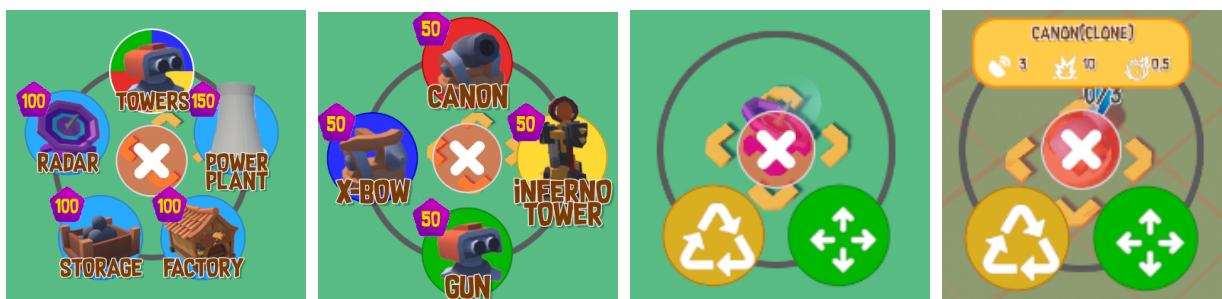
Ce HUD se décompose en 3 parties :

- La partie statique : elle comprend la barre de vie, le compte de l'argent et les trois boutons *Range Mode* qui permettent de changer la visualisation des portées (un mode portée d'attaque des tours, un mode portée d'alimentation pour les tours et centrales et portée des boost par les installations)



Visualisation des différentes portées

- La partie Préparation : Elle comprend le bouton pour lancer la vague de monstre, Un titre indiquant le numéro de la vague à venir, un menu roue pour placer les bâtiments, qui apparaît au clic sur un emplacement vide, et un autre menu roue pour déplacer ou supprimer un bâtiment, ou pour voir les statistiques d'une tour si le bâtiment visé en est une. Ce dernier menu n'apparaît qu'en cliquant sur un bâtiment.



Menu roue d'achat / déplacement / recyclage des bâtiments

- La partie Défense : Un titre indiquant le numéro de la vague en cours et une barre de progression qui se décharge à mesure que les ennemis sont éliminés. Cette progression est aussi indiquée par texte

L'interface comprend aussi un menu pause et un menu de fin de partie :

Le Menu pause est très simple, une fois affiché, celui-ci met en pause le jeu, les ennemis sont figés, les inputs bloqués etc. Il permet de quitter la partie ou de revenir dessus.



Menu pause

Le menu de fin de partie récapitule les statistiques de la partie qu'il récupère depuis le Script *GameStatsManager*. Ce menu aussi est très simple, on peut recommencer une partie sur cette carte ou retourner au menu principal.



Menu de fin de partie

Description globale de l'architecture de l'application.

L'application se décompose en deux scènes : Le menu principal et la scène de jeu.

Scène Menu Principal (MainMenu)

Il s'agit du point d'entrée dans l'application. Il permet de choisir la carte, et de lancer le jeu par la suite. On peut aussi y modifier les options du jeu (réglages audio, résolution, luminosité). Tout le menu est fait avec UI Toolkit.

Lors du lancement de la scène, AudioManager est instancié, et se charge des sons 2D de l'application. (Musique, clic sur les boutons...).

Lors de la sélection d'une carte dans la partie menu de sélection, des données nécessaires au lancement du jeu sont chargées dans la classe statique *GameConfig* (carte sous forme de *TileType*[], graphe des chemins de la carte et Asset de prefabs de carte), mais le jeu n'est pas lancé tout de suite. En effet, le chargement de ces données entraîne aussi la vérification de leur jouabilité (cf cahier des charges sur la vérification de la carte). Il faut ensuite cliquer sur le bouton START ! pour lancer la partie.

Scène de Jeu

La scène de jeu est plus complexe, elle est articulée autour de l'objet Game, porteur des scripts *Game* (pour gérer l'orchestration globale du jeu) et *MapGenerator* (qui sert à générer la carte) et ayant des enfants managers :

- HealthManager (*gérer la vie du joueur*)
- MoneyManager (*gérer l'argent*)
- WaveManager (*génération des vagues de monstres*)
- RangesManager (*affichage des portées des bâtiments*)
- BuildingPlacementManager (*gérer le placements des bâtiments*)
- GameStatsManager (*gérer les statistiques de la partie*)

Tous ces managers, ainsi que *Game*, sont des scripts singleton publiques, pour que leur données et méthodes publiques soient facilement accessibles depuis partout dans l'application, ce qui est pratique lors des calculs de répartition de l'énergie, des cibles pour les tours etc.

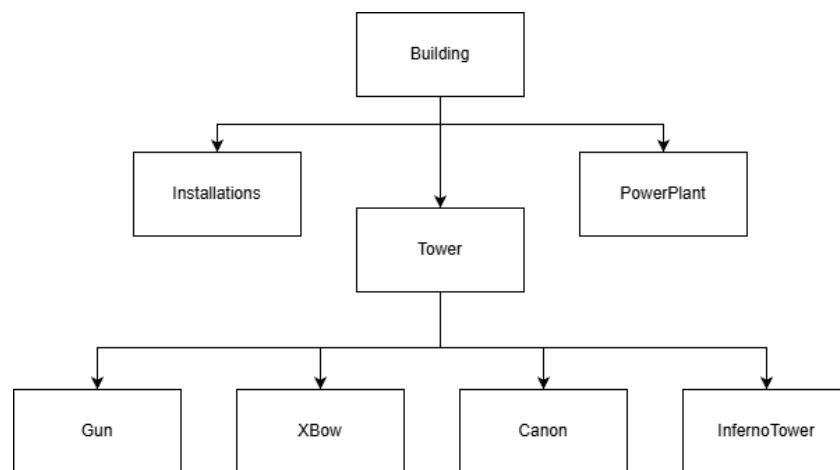
La scène contient aussi évidemment la caméra, avec son script *CameraControle* qui gère toutes les actions liées à la caméra. (mouvement, zoom, déplacement du curseur)

Au chargement, *Game* récupère les données précédemment stockées dans *GameConfig* par le menu pour générer la map proprement à l'aide de du script *MapGenerator*.

Pour que le jeu soit un jeu, il faut qu'il y ait de l'interaction entre le joueur et le jeu, le script *PlayerController* (singleton aussi) fait office d'interface pour le système d'Input.

Bâtiments

Les bâtiments sont liés par l'arbre d'héritage ci contre:



Arbre d'héritage simplifié des bâtiments

Chaque type de bâtiment possède ses propres caractéristiques modifiables et ajustables via des champs sérialisés. La liste des bâtiments actuellement présents et posés sur la carte est tenue par *Game* dans une liste publique.

Répartition du travail

Pour la répartition du travail, nous avons régulièrement ajouté des fonctionnalités à implémenter et chacun a codé celles qu'il voulait. Nous avons de ce fait chacun pu toucher à l'intégralité du projet pour autant travailler en même temps sur les mêmes fonctionnalités.

Bien que peu recommandable pour un projet professionnel, nous avons trouvé cette approche particulièrement intéressante, formatrice, et profitable sur le plan technique.

Antonin

1. Génération / Vérification de la carte

- a. Récupération de la carte dans le dossier Maps avec System.IO
- b. Traduction de l'image en Texture2D puis en *TileType*[][].
- c. Génération de la carte en tenant compte des rotations de certaines tuiles de chemin pour créer des chemins cohérents.
- d. Création des classes génériques *Graph<T>* et *Vertex<T>* pour la modélisation en graphe.
- e. Vérification de la carte avec *PathVerifier.cs* : algorithmes de parcours de graphe en profondeur, recherche de chemins.

2. Configurations de prefabs

Création de ScriptableObjects pour référencer des ensembles de prefabs cohérents.

- a. *BuildingsPrefabs* : références des prefabs de bâtiments.
- b. *MapPrefabs* : références des prefabs de la carte.
- c. *MonsterPrefabs* : références des prefabs de monster.
- d. Chaque tour a besoin de certains prefabs pour être comme elle est (particules, projectile, laser, ...)

3. Monstres

- a. recherche d'assets de monstres sur internet.
- b. écriture du script *MonsterController* pour gérer le comportement des monstres.
- c. animation des monstres avec les animations fournies dans les assets.
Créations d'animators avec des animations events pour le bon déroulé du jeu.
- d. Implémentation des comportements des monstres (*Strategies.cs*), plus court chemin ou plus court chemin + recouvrement.

4. Energie

- a. Création du graphe d'énergie
- b. Attribution de l'énergie entre les tours
- c. Restriction des capacité des tours en fonction de leur alimentation.
- d. Visualisation des liens entre PowerPlant et tours avec des lignes bleues
- e. Visualisation de la puissance reçue par les tours à l'aide d'UIDocument intégrés aux prefabs

5. Patron des script Building et Tower

- a. En commençant à implémenter une tour je me suis rendu compte que toutes les tours n'allait pas marcher sensiblement de la même manière. Il a donc fallu regrouper le commun dans *Tower* et raffiner dans des classes héritant de *Tower*.
- b. Dans cette même logique, j'ai pensé que quand mes camarades commenceront à créer les autres bâtiments, ils auront besoin de certaines fonctionnalités propres à *Tower*, d'où l'apparition de la classe *Building*.
- c. Cette hiérarchie permet un bon niveau d'abstraction très pratique dans ce type de projet.
- d. Création des tours Gun et Canon depuis le pack d'assets fourni dans le pdf du projet.
- e. Création de la centrale (PowerPlant) à partir d'un modèle 3D trouvé sur internet. Ajout de fumée pour l'esthétisme.

6. UI de la partie Jeu

- a. Création de tous les éléments statiques visibles à l'écran avec UIToolkit
- b. Création d'images personnalisées avec un éditeur d'image externe, des choses simples comme les barres de vie et de progression, le pentagone de l'argent, le bouton "FIGHT !", les boutons pour sélectionner le type de range qu'on veut voir au sol et le détournage des bâtiments dans le menu roue d'achat.
- c. Création de toute la logique derrière l'UI.

7. Déroulement du jeu

- a. Écriture de *Game*
 - i. mise à disposition des données nécessaire au fonctionnement global du jeu (état du jeu, monstres et bâtiments présent dans la scène, sélecteur, assets de prefabs, référence vers le *HUDManager*

- ii. Indique quoi faire dans quel état.
- b. Écriture de *WaveManager*
 - i. Créer des vagues selon un pattern déterministe
 - ii. Instancier les monstres sur un point de spawn aléatoire
 - iii. Refait passer l'état du jeu a Préparation une fois la vague morte.

8. Particules / FX

- a. Ajout de particules dans le projet pour de l'esthétisme
- b. Spawn d'une vague
- c. Monstre touché
- d. Monstre mort
- e. Crystal attaqué
- f. Tours qui tirent (canon → explosion par exemple)

Alexandre

1. Implémentation des Installations de Soutien

Conformément au Game Design Document, j'ai développé le système d'installations (Radar, Usine d'armement, Stock de munitions) visant à améliorer les performances des tours défensives.

- a. Conception visuelle et Assets : Création et optimisation des modèles 3D pour assurer une cohérence graphique avec l'ensemble du projet.
- b. Effets de Particules (FX) : Pour améliorer le feedback visuel (Game Feel), j'ai intégré un Particle System sur le Radar simulant une onde électromagnétique lors de son déploiement sur la carte.
- c. Gestion de la Portée : Mise en place d'un système de zone d'effet pour appliquer les bonus de statistiques (portée, puissance ou cadence) aux tours situées dans le rayon d'action du bâtiment.

2. Système d'économie et équilibrage

J'ai conçu l'intégralité de la boucle économique, essentielle pour la progression dans le mode "survie".

- a. Cycle d'Achat/Revente : Implémentation d'un système de gestion de ressources permettant l'acquisition de nouveaux bâtiments et la revente de structures existantes pour ajuster la stratégie en temps réel.
- b. Récompenses de Combat : Intégration d'un trigger lors de la destruction des monstres pour créditer automatiquement le compte du joueur.
- c. Architecture Flexible (Système d'Enum) : Afin de faciliter l'équilibrage par l'équipe, j'ai créé un système basé sur des Enums et des variables sérialisées. Cela permet de modifier le coût de chaque bâtiment directement depuis l'inspecteur Unity sans modifier le code source.
- d. Interface Utilisateur (HUD) : Développement d'un affichage dynamique indiquant en temps réel la somme totale possédée et le coût des bâtiments dans le menu de construction.

3. Fonctionnalités de navigation et paramètres

- a. Contrôle de la caméra : En complément du déplacement clavier (Z,Q,S,D), j'ai implémenté le système de zoom et dézoom à la molette, offrant une meilleure lecture tactique de la carte.
- b. Paramètres Système (Persistance) : Gestion de la modification de la résolution d'affichage depuis le menu Settings, avec une sauvegarde des préférences utilisateur via l'API PlayerPrefs d'Unity.

4. Audio

- a. Ajout des SFX sur les monstres (spawn, hit et mort).

Julien

1. Menu Settings

- a. **Luminosité** : Le premier slider correspond à la luminosité (*brightness*), le concept est juste de faire apparaître une voile noire avec une opacité proportionnelles à la valeur du slider
- b. **Audio** : Création de deux audiosources, un pour les FX et un autre pour la musique ambiante. On drag and drop ensuite les fichiers mp3 sur les audioclip qu'on fait ensuite jouer sur les audiosources selon son type. Les 3 sliders suivants dans le menu SETTINGS correspondent respectivement au FX, Musique et le Master. Le AudioManager est en *DontDestroyOnLoad* pour que les sons puissent marcher peu importe quelle scène.

2. Partie jeu

- a. **Contrôle de la caméra** : Je me suis énormément inspiré de ce qui est fait dans les différents TP où on doit contrôler une caméra (*InputAction*). Et de même pour le zoom.
- b. Les **tours rouge et verte** sont développées par Antonin. La **tour bleue** est une variante directe de ces dernières : elle partage la même logique de ciblage unique (vise le premier ennemi qui rentre dans sa zone et change de target si le monstre meurt ou si le monstre sort de la range, *Mono-Targeting*), mais se distingue par un visuel différent et des caractéristiques différentes. La **tour jaune**, quant à elle, introduit une stratégie de zone (*Multi-Targeting*) en attaquant tous les ennemis à sa portée. Son intégration a nécessité un développement spécifique : inspirée de la tour de l'enfer de Clash of Clan, elle est statique (aucune rotation) et inflige des dégâts instantanés sans projectile, ce qui a demandé une adaptation du code.
- c. Ajout d'un menu pause. Le menu pause s'active lorsqu'on appuie sur "echap". Le jeu s'assombrit légèrement puis deux boutons sont offerts à nous. "Resume" et "Quit". Le premier pour reprendre la partie et le deuxième pour quitter la partie en cours et nous emmener sur le menu principal.
- d. La première version du menu mort. Quand le point de vie du joueur atteint 0, l'écran de mort s'affiche et il nous propose de soit relancer la partie en jouant sur la même carte, soit de revenir sur le menu principal.

Difficultés rencontrées

Le temps

Comme tous les groupes, le facteur temps est le plus gros limiteur dans ce type de projet. Les consignes sont nombreuses et précises et on peut vite s'y perdre avec ce qu'on devait faire et ce qu'on souhaitait inclure dans notre projet pour le rendre plus personnel.

Mauvais choix (RigidBody)

Pour trois des quatre tours nous avons très tôt décidé, par soucis de simplicité d'implémentation, d'utiliser des rigidbody comme projectiles. Évidemment ce choix n'était pas le bon car les tours ratent de manière récurrente leur cible.

Manque d'expérience

Le projet dans son ensemble est fonctionnel comme nous le voulons mais nous sommes persuadés que beaucoup de script ou d'éléments ne respectent pas toutes les normes de programmation. Par exemple la bonne séparation des éléments entre eux, les managers qui ne s'occupent que de leurs affaires..etc. C'est un point difficile à expliquer mais nous sentons que le code ici fourni est très perfectible sur ce point là. Cela est largement dû à un manque d'expérience en programmation orienté objet dans ce contexte de "gros" projet dans un premier temps, puis un manque d'expérience avec l'outil Unity dans un second.

Le code a beau être fonctionnel, nous ne sommes pas 100% fiers de toutes les méthodes employées pour y arriver.

Équilibrage du jeu

Nous ne nous sommes que très peu attardé sur la partie équilibrage du jeu. Celle-ci est pourtant cruciale pour qu'un jeu soit intéressant à jouer. Il en résulte que l'Arc-X et la tour

de l'enfer (moyennant des boost de portée) sont suffisants pour gagner tout le temps. On gagne aussi beaucoup trop d'argent par rapport aux dépenses.

Résultats

Performances

En jouant au jeu, on se rend compte que le jeu rame un petit peu très rapidement quand le nombre d'objets présents dans la scène devient trop grand. (l'utilisation de *System.Linq*, bien que très pratique pour faire des recherches dans des listes, pourrait aussi être très lourde en calculs)

Idées d'améliorations :

- Beaucoup de `List<T>` sont utilisées, le débat `List<T>` ou `HashSet<T>` pourrait exister à certains endroits du code.
- Les bordures sont générées avec des tuiles comme l'est la carte, ce qui prend très probablement beaucoup d'espace mémoire.

Conclusion

Ce projet a été un projet à la fois très chronophage, très formateur, mais aussi très amusant. Il nous a permis d'approfondir et de personnaliser grandement les compétences acquises en cours, et de les mettre à profit dans un projet à notre image.